CSS183-2 SA02
Group 19:
NOVERO, Karrin Frida
BALAGAO, Aleah

Part 1: Environment Setup

1. Mount Google Drive

This step connects Google Drive to the Colab environment using drive.mount(). It allows access to the dataset and data.yaml file stored in Drive. Without mounting, the model cannot locate the training and validation images.

The pip install ultralytics command installs the YOLO framework. This ensures that the correct version of the model is available for training.

```
from google.colab import drive
drive.mount('/content/drive')
```

```
Mounted at /content/drive
```

```
!pip install ultralytics

import ultralytics
print(ultralytics.__version__)
```

```
Downloading ultralytics_thop-2.0.18-py3-none-any.whl (28 kB)
Installing collected packages: ultralytics-thop, ultralytics
Successfully installed ultralytics-8.4.18 ultralytics-thop-2.0.18
Creating new Ultralytics Settings v0.0.6 file ✅
View Ultralytics Settings with 'yolo settings' or at '/root/.config/Ultralytics/settings.json'
Update Settings with 'yolo settings key=value', i.e. 'yolo settings runs_dir=path/to/dir'. For help see https://doc
8.4.18
```

2. Load the dataset

The data.yaml file defines the dataset configuration. It includes the paths for training and validation images, the number of classes, and class names such as fire and smoke. This file is passed into the model so it knows how to read the dataset.

```
!ls /content/drive/MyDrive/GRP19_SA1/'Annotated Datasets'/SA01_Annotations.v2-final.yolov8/data.yaml
```

```
'/content/drive/MyDrive/GRP19_SA1/Annotated Datasets/SA01_Annotations.v2-final.yolov8/data.yaml'
```

```
!cat "/content/drive/MyDrive/GRP19_SA1/Annotated Datasets/SA01_Annotations.v2-final.yolov8/data.yaml"
```

```
train: ../train/images
val: ../valid/images
test: ../test/images

nc: 6
names: ['Bus', 'Motorcycle', 'a', 'car', 'person', 'truck']

roboflow:
  workspace: karrins-workspace
  project: sa01_annotations
  version: 2
  license: CC BY 4.0
  url: https://universe.roboflow.com/karrins-workspace/sa01_annotations/dataset/2
```

Part 2: Model Initialization and Training

1. Import YOLO and check GPU status

The command import torch loads the PyTorch library, which is used by YOLO for deep learning operations. The function torch.cuda.is_available() checks whether a GPU is accessible in the current environment.

```
[ ]
from ultralytics import YOLO
```

```
[ ]
import torch
print(torch.cuda.is_available())
```

```
True
```

2. Model Initialization

The command YOLO("yolov8n.pt") loads the pre-trained nano version of YOLO. The nano model is lightweight and suitable for faster training. This step prepares the model architecture for fine-tuning on the custom dataset.

3. Model Training

The model.train() function begins the training process. Parameters such as epochs, batch size, image size, optimizer, and learning rate control how the model learns. During training, the system calculates box loss, classification loss, and detection performance. The results are automatically saved in the runs directory.

This is the first model.

```
model1 = YOLO("yolo26n.pt")

results1 = model1.train(
    data="/content/drive/MyDrive/GRP19_SA1/Annotated Datasets/SA01_Annotations.v2-final.yolov8/data.yaml",
    epochs=25,
    imgsz=640,
    batch=4,
    optimizer="AdamW",
    lr0=0.01
)
```

```
25 epochs completed in 0.430 hours.
Optimizer stripped from /content/runs/detect/train2/weights/last.pt, 5.4MB
Optimizer stripped from /content/runs/detect/train2/weights/best.pt, 5.4MB

Validating /content/runs/detect/train2/weights/best.pt...
Ultralytics 8.4.18 🚀 Python-3.12.12 torch-2.10.0+cu128 CUDA:0 (Tesla T4, 14913MiB)
YOLO26n summary (fused): 122 layers, 2,376,006 parameters, 0 gradients, 5.2 GFLOPs
                 Class     Images  Instances      Box(P          R      mAP50  mAP50-95): 100% ━━━━━━━━━━━━ 18/18
                   all        141       1441      0.658       0.64      0.675      0.512
                   Bus         17         35      0.606      0.686      0.774      0.612
            Motorcycle         22         47      0.605      0.383      0.409      0.275
                   car        119       1184      0.868      0.865      0.922      0.787
                person         40         65      0.562      0.523      0.488      0.261
                 truck         67        110      0.649      0.745      0.781      0.624
Speed: 0.6ms preprocess, 10.6ms inference, 0.0ms loss, 0.7ms postprocess per image
Results saved to /content/runs/detect/train2
```

This is the second model.

```
model2 = YOLO("yolo26n.pt")

results2 = model2.train(
    data="/content/drive/MyDrive/GRP19_SA1/Annotated Datasets/SA01_Annotations.v2-final.yolov8/data.yaml",
    epochs=30,
    imgsz=640,
    batch=20,
    optimizer="SGD",
    lr0=0.001
)
```

```
30 epochs completed in 0.282 hours.
Optimizer stripped from /content/runs/detect/train3/weights/last.pt, 5.4MB
Optimizer stripped from /content/runs/detect/train3/weights/best.pt, 5.4MB

Validating /content/runs/detect/train3/weights/best.pt...
Ultralytics 8.4.18 🚀 Python-3.12.12 torch-2.10.0+cu128 CUDA:0 (Tesla T4, 14913MiB)
YOLO26n summary (fused): 122 layers, 2,376,006 parameters, 0 gradients, 5.2 GFLOPs
                 Class     Images  Instances      Box(P          R      mAP50  mAP50-95): 100% ━━━━━━━━━━━━ 4/4 1.
                   all        141       1441      0.592      0.497      0.547      0.432
                   Bus         17         35      0.638      0.514      0.605      0.525
            Motorcycle         22         47      0.553      0.277      0.331      0.239
                   car        119       1184       0.71      0.833      0.838      0.689
                person         40         65      0.523       0.27      0.412      0.239
                 truck         67        110      0.534      0.591       0.55      0.469
Speed: 0.2ms preprocess, 3.2ms inference, 0.0ms loss, 1.1ms postprocess per image
Results saved to /content/runs/detect/train3
```

This is the third model.

```
model3 = YOLO("yolo26n.pt")

results3 = model3.train(
    data="/content/drive/MyDrive/GRP19_SA1/Annotated Datasets/SA01_Annotations.v2-final.yolov8/data.yaml",
    epochs=40,
    imgsz=640,
    batch=-1,
    optimizer="auto",
    lr0=0.0001
)
```

```
40 epochs completed in 0.383 hours.
Optimizer stripped from /content/runs/detect/train4/weights/last.pt, 5.4MB
Optimizer stripped from /content/runs/detect/train4/weights/best.pt, 5.4MB

Validating /content/runs/detect/train4/weights/best.pt...
Ultralytics 8.4.18 🚀 Python-3.12.12 torch-2.10.0+cu128 CUDA:0 (Tesla T4, 14913MiB)
YOLO26n summary (fused): 122 layers, 2,376,006 parameters, 0 gradients, 5.2 GFLOPs
                 Class     Images  Instances      Box(P          R      mAP50  mAP50-95): 100% ━━━━━━━━━━━━ 4/4 1
                   all        141       1441      0.719       0.71      0.744      0.606
                   Bus         17         35       0.84      0.857      0.913      0.764
            Motorcycle         22         47      0.499      0.447      0.446      0.318
                   car        119       1184      0.851      0.871      0.925      0.797
                person         40         65      0.645        0.6      0.584      0.401
                 truck         67        110      0.761      0.773      0.852      0.749
Speed: 0.3ms preprocess, 3.2ms inference, 0.0ms loss, 0.8ms postprocess per image
Results saved to /content/runs/detect/train4
```

Part 3: Confusion Matrix

The function os.listdir views the training folders, including train, train 2, and more. This step is taken to find the location of the outputs, such as results and confusion matrix.

```
import os

print(os.listdir("/content/runs/detect"))

print(os.listdir("/content/runs/detect/train"))
print(os.listdir("/content/runs/detect/train2"))
```

```
['train4', 'train3', 'train', 'train2']
['train_batch2.jpg', 'train_batch0.jpg', 'labels.jpg', 'train_batch1.jpg', 'args.yaml', 'weights']
['BoxP_curve.png', 'val_batch1_pred.jpg', 'val_batch0_pred.jpg', 'results.csv', 'confusion_matrix.png', '
```

The confusion_matrix.png file is generated after validation. The matrix compares actual classes versus predicted classes. The diagonal values represent True Positives, meaning correct predictions. The off-diagonal values represent False Positives and False Negatives, meaning incorrect classifications.

1. Confusion Matrix in train run folder 2

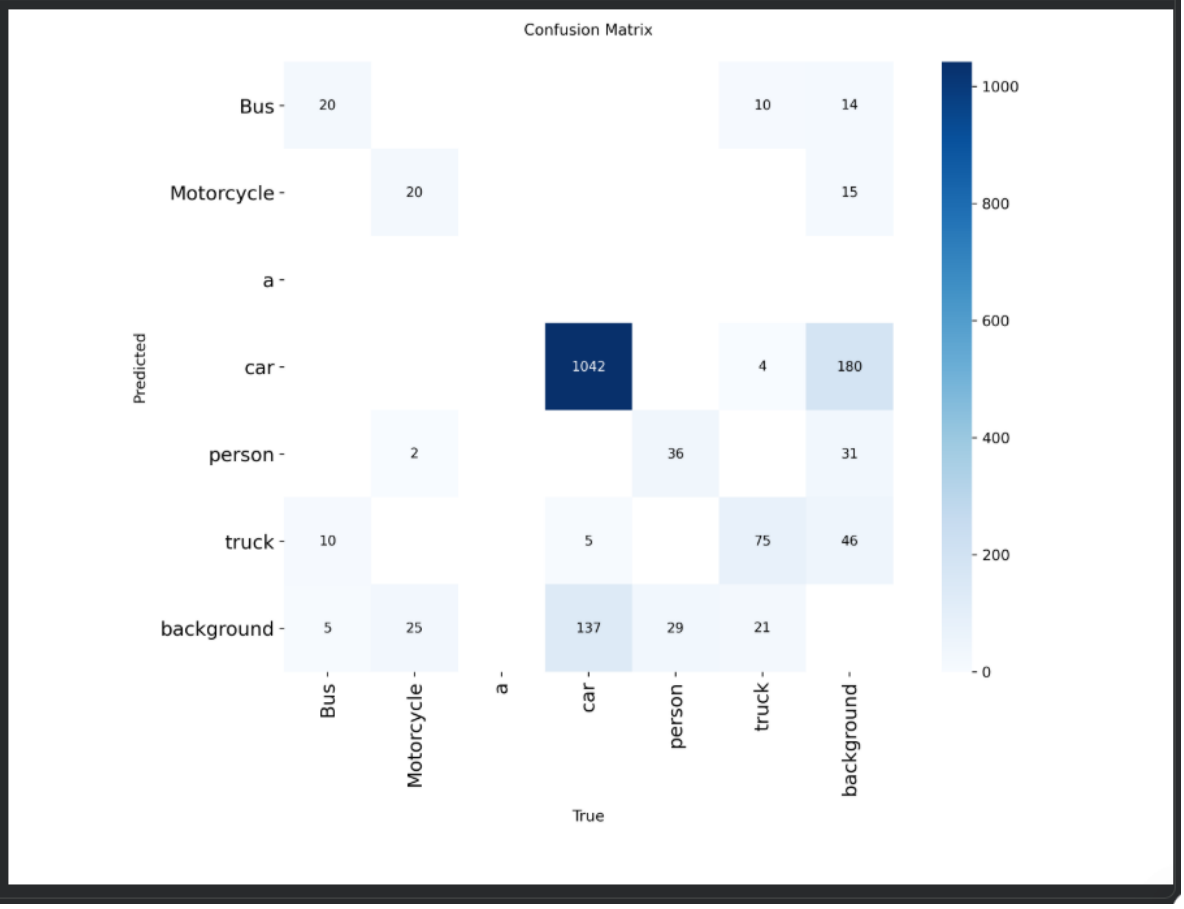
The true negatives are excluded since they are not directly used in YOLO models. Following the logic that the y-axis contains the predicted values and the x-axis contains the true values, then the following is a list of the values:

1. True Positive

    a. Bus - 20
    b. Motorcycle - 20
    c. Car - 1042
    d. Person - 36
    e. Truck - 75
    f. Background - 0
2. False Positive
    a. Bus - 24
    b. Motorcycle - 15
    c. Car - 184
    d. Person - 33
    e. Truck - 61
    f. Background - 217
3. False Negative
    a. Bus - 15
    b. Motorcycle - 27
    c. Car - 142
    d. Person - 29
    e. Truck - 35
    f. Background - 276

```
from IPython.display import Image

Image("/content/runs/detect/train2/confusion_matrix.png")
```



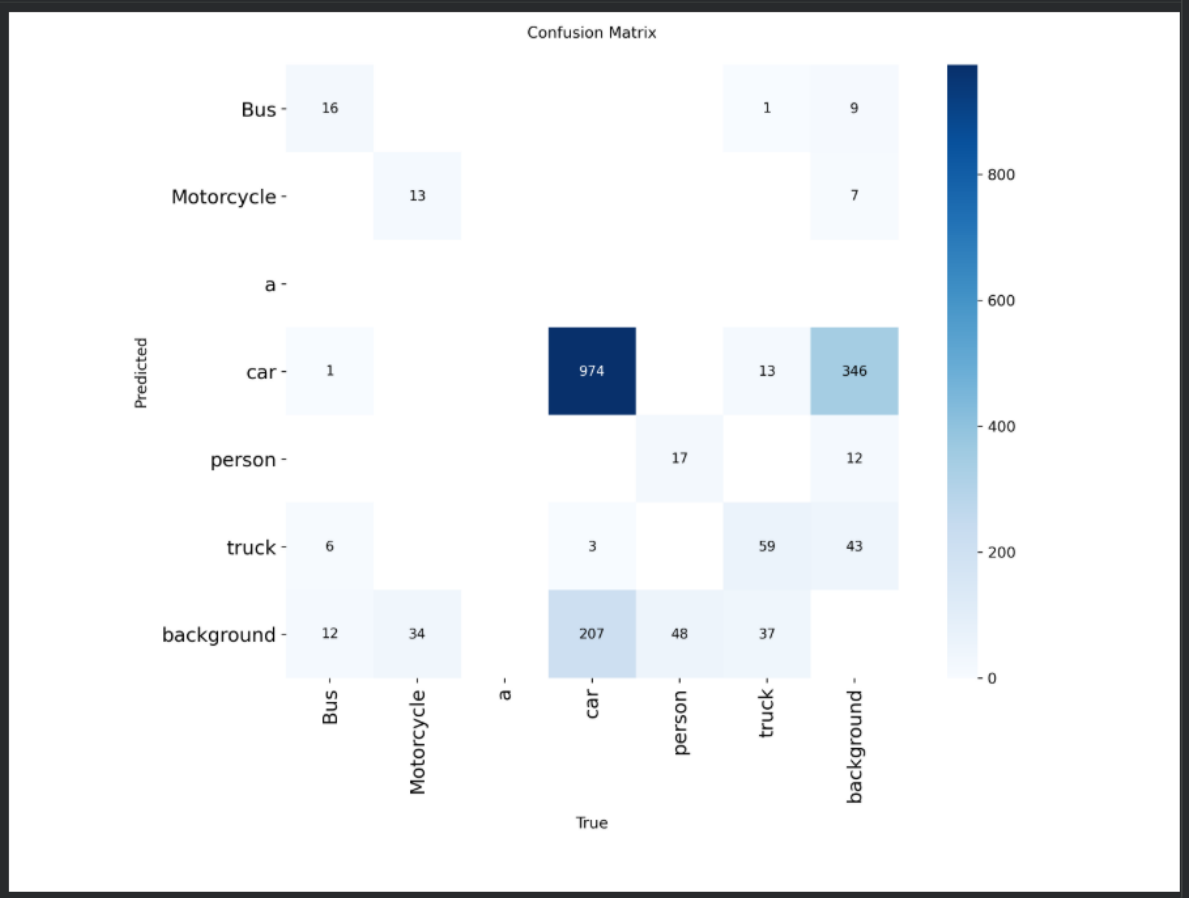
2. Confusion Matrix in train run folder 3

The true negatives are excluded since they are not directly used in YOLO models. Following the logic that the y-axis contains the predicted values and the x-axis contains the true values, then the following is a list of the values:

1. True Positive
    a. Bus - 16
    b. Motorcycle - 13
    c. Car - 974
    d. Person - 17
    e. Truck - 59
    f. Background - 0
4. False Positive
    a. Bus - 10
    b. Motorcycle - 7
    c. Car - 360
    d. Person - 12

    e. Truck - 52
    f. Background - 338
5. False Negative
    a. Bus - 19
    b. Motorcycle - 34
    c. Car - 210
    d. Person - 48
    e. Truck - 51
    f. Background - 417

```
Image("/content/runs/detect/train3/confusion_matrix.png")
```



Part 4: Model Validation

This step runs the validation on the trained models. The command .val() checks how well the model detects objects in the validation dataset. It calculates metrics like Precision, Recall, mAP50, and mAP50-95 for each class. The results are stored in metrics1, metrics2, and metrics3 for Model 1, Model 2, and Model 3, respectively. These objects are used to see how accurate each model is and which classes it struggles with.

```
metrics1 = model1.val()
metrics2 = model2.val()
metrics3 = model3.val()

Ultralytics 8.4.18 🚀 Python-3.12.12 torch-2.10.0+cu128 CUDA:0 (Tesla T4, 14913MiB)
YOLO26n summary (fused): 122 layers, 2,376,006 parameters, 0 gradients, 5.2 GFLOPs
val: Fast image access ✅ (ping: 0.9±0.4 ms, read: 17.3±9.9 MB/s, size: 45.3 KB)
val: Scanning /content/drive/.shortcut-targets-by-id/1LZeM4mKs9FFHA_foRestzdO31Eb5dcu_/GRP19_SA01/Annotated Datasets/
                 Class     Images  Instances      Box(P          R      mAP50  mAP50-95): 100% ━━━━━━━━━━━━ 9/9 1.4it
                   all        141       1441      0.656      0.641      0.674       0.51
                   Bus         17         35      0.605      0.686       0.77        0.6
            Motorcycle         22         47      0.605      0.383      0.409      0.275
                   car        119       1184      0.869      0.865      0.922      0.788
                person         40         65      0.554      0.523      0.487      0.261
                 truck         67        110      0.648      0.745      0.781      0.624
Speed: 6.2ms preprocess, 10.4ms inference, 0.0ms loss, 1.0ms postprocess per image
Results saved to /content/runs/detect/val
Ultralytics 8.4.18 🚀 Python-3.12.12 torch-2.10.0+cu128 CUDA:0 (Tesla T4, 14913MiB)
YOLO26n summary (fused): 122 layers, 2,376,006 parameters, 0 gradients, 5.2 GFLOPs
val: Fast image access ✅ (ping: 0.8±0.3 ms, read: 19.3±6.0 MB/s, size: 43.1 KB)
val: Scanning /content/drive/.shortcut-targets-by-id/1LZeM4mKs9FFHA_foRestzdO31Eb5dcu_/GRP19_SA01/Annotated Datasets/
                 Class     Images  Instances      Box(P          R      mAP50  mAP50-95): 100% ━━━━━━━━━━━━ 9/9 1.8it
                   all        141       1441      0.592      0.493      0.547      0.432
                   Bus         17         35      0.639      0.514      0.605      0.525
            Motorcycle         22         47      0.555      0.277      0.331      0.239
                   car        119       1184      0.714      0.832      0.838      0.689
                person         40         65      0.519      0.266      0.412      0.239
                 truck         67        110      0.531      0.575       0.55       0.47
Speed: 6.0ms preprocess, 5.7ms inference, 0.0ms loss, 0.4ms postprocess per image
Results saved to /content/runs/detect/val2
Ultralytics 8.4.18 🚀 Python-3.12.12 torch-2.10.0+cu128 CUDA:0 (Tesla T4, 14913MiB)
YOLO26n summary (fused): 122 layers, 2,376,006 parameters, 0 gradients, 5.2 GFLOPs
val: Fast image access ✅ (ping: 0.7±0.4 ms, read: 17.3±6.1 MB/s, size: 42.1 KB)
val: Scanning /content/drive/.shortcut-targets-by-id/1LZeM4mKs9FFHA_foRestzdO31Eb5dcu_/GRP19_SA01/Annotated Datasets/
                 Class     Images  Instances      Box(P          R      mAP50  mAP50-95): 100% ━━━━━━━━━━━━ 9/9 1.5it
                   all        141       1441       0.72      0.711      0.744      0.605
                   Bus         17         35      0.839      0.857      0.912      0.758
            Motorcycle         22         47      0.499      0.447      0.446      0.321
                   car        119       1184      0.851      0.872      0.925      0.796
                person         40         65      0.653      0.608      0.584      0.397
                 truck         67        110      0.759      0.773      0.852      0.754
Speed: 8.0ms preprocess, 6.7ms inference, 0.0ms loss, 0.6ms postprocess per image
Results saved to /content/runs/detect/val3
```

Part 6: Tables

1. Hyperparameter Table

This table is a record of the parameters used in each model.

```python
import pandas as pd

hyper_table = pd.DataFrame({
    "Model": ["Model 1", "Model 2", "Model 3"],
    "Epochs": [25, 30, 40],
    "Optimizer": ["AdamW", "SGD", "auto"],
    "Batch Size": [4, 20, -1],
    "Learning Rate": [0.01, 0.001, 0.0001],
    "Image Size": [640, 640, 640]
})

hyper_table
```

	Model	Epochs	Optimizer	Batch Size	Learning Rate	Image Size
0	Model 1	25	AdamW	4	0.0100	640
1	Model 2	30	SGD	20	0.0010	640
2	Model 3	40	auto	-1	0.0001	640

2. Metrics Table

Pandas is used to create a formatted table of hyperparameter settings for each model. Another table displays mAP50, Precision, Recall, and F1 Score. These tables allow comparison of model performance to determine the best configuration. The columns include mAP50, Precision, Recall, and F1 Score, whose values are taken from metrics1, metrics2, and metrics3.

```python
metrics_table = pd.DataFrame({
    "Model": ["Model 1", "Model 2", "Model 3"],
    "mAP50": [
        metrics1.box.map50,
        metrics2.box.map50,
        metrics3.box.map50
    ],
    "Precision": [
        metrics1.box.mp,
        metrics2.box.mp,
        metrics3.box.mp
    ],
    "Recall": [
        metrics1.box.mr,
        metrics2.box.mr,
        metrics3.box.mr
    ],
    "F1 Score": [
        2*(metrics1.box.mp*metrics1.box.mr)/(metrics1.box.mp+metrics1.box.mr),
        2*(metrics2.box.mp*metrics2.box.mr)/(metrics2.box.mp+metrics2.box.mr),
        2*(metrics3.box.mp*metrics3.box.mr)/(metrics3.box.mp+metrics3.box.mr)
    ]
})

metrics_table
```

	Model	mAP50	Precision	Recall	F1 Score
0	Model 1	0.673655	0.655980	0.640535	0.648166
1	Model 2	0.547269	0.591500	0.492703	0.537600
2	Model 3	0.743601	0.720211	0.711280	0.715718

Part 7: Discussion of Model Performance

1. Class Performance

The first confusion matrix contains the distribution of correct and incorrect predictions as well as the actual values of the model. The true negatives are excluded since they are not directly used in YOLO models. For Bus, there are 20 true positives, 24 false positives, and 15 false negatives, showing decentdetection but a notable amount of misclassification. For the motorcycle, the model correctly predicted 20 instances but missed 27, which can mean a weak recall.

For Car, there are 1042 true positives therefore it displays the strongest detection performance. For Person, 36 true positives with similar false positives and false negatives shows a moderate performance. For Truck, 75 true positives show good detection, but still some confusion with other vehicles. Additionally, all three models are unable to detect the background accurately. Overall,

this run performs well on large vehicles like cars and trucks but struggles with smaller objects such as motorcycles and persons.

There is a noticeable drop in accurate predictions in the second confusion matrix. There is an indirect relationship because as the false positives and false negatives increase, then the true positives decrease. The detection and classification for buses and motorcycles decreased, meaning there's a lower recall. In cars, there's a rise in false positives since the model is struggling to accurately classify objects. Person and Truck classes show reduced true positives and increased missed detections. It is clear that the parameters play a crucial role in training the models. Overall, this run demonstrates weaker performance and more misclassification across all classes.

2. Hyperparameter Effects

The three models were trained with different hyperparameters, including epochs, optimizer, batch size, and learning rate. Model 1 was trained for 25 epochs using AdamW with a batch size of 4 and learning rate of 0.01. It achieved an mAP50 of 0.674, a precision of 0.656, a recall of 0.641, and an F1 score of 0.648. This means that the results were balanced but not fully accurate.

Model 2 was trained for 30 epochs using SGD with batch size 20 and learning rate of 0.001. It achieved the lowest performance with mAP50 of 0.547, precision of 0.592, recall of 0.493, and F1 score of 0.538. The low recall indicates that many objects were missed. Among the models, this has missed the most objects and misclassified categories.

Model 3 was trained for 40 epochs using the auto optimizer with automatic batch size and learning rate of 0.0001. It achieved the highest performance with mAP50 of 0.744, precision of 0.720, recall of 0.711, and F1 score of 0.716. There is a better balance between detection and classification. Increasing the number of epochs improved learning and allowed the model to converge better. The smaller learning rate in Model 3 helped stabilize training and reduce large weight updates.

3. Overall Evaluation

The confusion matrix and validation metrics show that Model 3 performs the best overall. It shows more true positives and fewer misclassifications than the other models. Car detection is the most reliable of all the models due to the high volume of occurrences and more pronounced visual features. Motorcycles and people remain the weakest classes because to their smaller item sizes and more visual resemblance to other classes. In practical scenarios, Model 3 exhibits the greatest promise.

References:
https://docs.ultralytics.com/models/yolo26/#usage-examples

https://learnopencv.com/yolov26-real-time-deployment/#aioseo-yoloe-26-open-vocabulary-instance-segmentation-56
https://github.com/ultralytics/ultralytics?tab=readme-ov-file&ref=blog.roboflow.com#-models
https://www.youtube.com/watch?v=7lZa3Yi2kbo
https://medium.com/@zainshariff6506/a-simple-yolo26-tutorial-from-beginners-to-experts-b5aa491b8ace
https://learnopencv.com/yolo26-nms-free-inference/